Prinzipien der Robotik

Embedded Systems Engineering – Vorlesung

Prof. Dr.-Ing. Sebastian Schlesinger

Fachbereich 2 – Duales Studium Wirtschaft & Technik
HWR Berlin

Wintersemester 2026

Gliederung

1 Sensorsignale verstehen

2 Signalverarbeitung

3 Zustandsschätzung

4 Kinematik & Bewegungsmodelle

5 Regelung (Control)

6 Lokalisierung & Mapping (SLAM)

7 Pfad- und Trajektorienplanung

8 Softwarearchitektur (ROS 2)

9 Echtzeit und Embedded-Aspekte

10 Unsicherheit & Verifikation

11 Zusammenfassung

Von der Modellierung zur Implementierung

Bisherige Vorlesung: Werkzeuge und Methoden der Modellierung

- Software Engineering, FSMs, State Charts
- Timed Automata, Verifikation mit UPPAAL
- Programmierung in C und Rust
- ROS 2 als Middleware

Von der Modellierung zur Implementierung

Bisherige Vorlesung: Werkzeuge und Methoden der Modellierung

- Software Engineering, FSMs, State Charts
- Timed Automata, Verifikation mit UPPAAL
- Programmierung in C und Rust
- ROS 2 als Middleware

Jetzt: Wie setzt man diese Werkzeuge ein, um einen realen Roboter zu bauen und zu steuern?

Von der Modellierung zur Implementierung

Bisherige Vorlesung: Werkzeuge und Methoden der Modellierung

- Software Engineering, FSMs, State Charts
- Timed Automata, Verifikation mit UPPAAL
- Programmierung in C und Rust
- ROS 2 als Middleware

Jetzt: Wie setzt man diese Werkzeuge ein, um einen realen Roboter zu bauen und zu steuern?

Kernfrage

Wie kommt ein mobiler Roboter von Sensordaten zu sicherer, zielgerichteter Bewegung – und wie verifizieren wir das?

Sensorsignale verstehen

Sensormodelle – Überblick

Ein Roboter nimmt die Welt ausschließlich über Sensoren wahr. Jeder Sensor liefert ein *Modell* der physikalischen Größe.

Inertialsensoren (IMU)

- Beschleunigungssensor: $a = F/m$
- Gyroskop: Winkelgeschwindigkeit ω
- 6-DOF (degrees of freedom): $[a_x, a_y, a_z, \omega_x, \omega_y, \omega_z]^T$

a_i ist die Beschleunigung entlang der i-ten Achse (Bewegung im Raum), ω_i die Winkelgeschwindigkeit um die i-te Achse (Drehung im Raum).

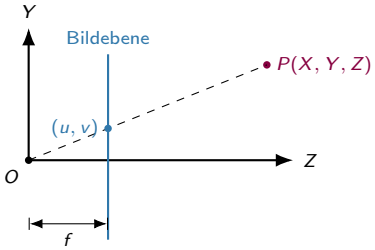
Entfernungssensoren

- Lidar / Ultraschall: Time-of-Flight
- $d = \frac{c \cdot \Delta t}{2}$
- Lidar: Laserpulse, 360-Scan
- Ultraschall: Schallkegel, geringere Auflösung

Sensormodell: Kamera – Projektionsmodell

Pinhole-Kameramodell: $(u, v, w) = K \cdot (X, Y, Z)$
(homogene Koordinaten) $\Rightarrow$ 2D-Bildkoordinaten
durch Normalisierung: $(u/w, v/w)$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z} \underbrace{\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{K \text{ (Kameramatrix)}} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$



- f_x, f_y: Brennweite in Pixel
- c_x, c_y: Bildhauptpunkt
- 3D-Punkt $(X, Y, Z) \mapsto$ 2D-Pixel (u, v)

Signalcharakteristika

Kein Sensor liefert perfekte Daten. Typische Störeinflüsse:

Rauschen

- Gaussian Noise: $n \sim \mathcal{N}(0, \sigma^2)$
- Signal-Rausch-Verhältnis (SNR)
- Thermisches Rauschen, Quantisierungsrauschen

Bias & Drift

- Konstanter Offset: $\hat{x} = x + b$
- Drift: $b(t)$ ändert sich langsam über Zeit
- Besonders kritisch bei Gyroskopen (IMU)

Quantisierung (ADC)

- Analog $\rightarrow$ Digital
- n Bit $\Rightarrow 2^n$ Stufen
- Auflösung: $\Delta = \frac{V_{\text{ref}}}{2^n}$
- 10-Bit ADC bei 3.3 V: $\Delta \approx 3.2\,\text{mV}$

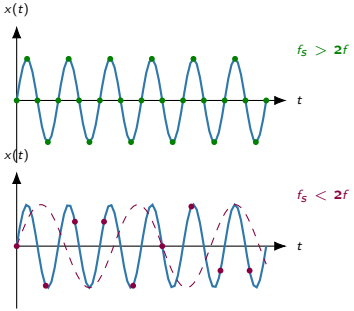
Abtasttheorem

Ein Signal mit maximaler Frequenz $f_{\max}$ kann exakt rekonstruiert werden, wenn die Abtastrate $f_s > 2 \cdot f_{\max}$ beträgt.

Aliasing: Wird zu langsam abgetastet ($f_s \leq 2f_{\max}$), erscheinen hochfrequente Komponenten als niederfrequente Geistersignale.

Praxis:

- IMU: typisch 100 Hz – 1000 Hz
- Lidar: 5 Hz – 40 Hz (Scanrate)
- Kamera: 30 Hz – 60 Hz (Framerate)



Signalverarbeitung

Filter – Grundlagen

Rohe Sensordaten sind verrauscht $\Rightarrow$ Filterung nötig.

Moving Average (Gleitender Mittelwert):

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k]$$

- Einfach zu implementieren
- Glättet Rauschen
- Verzögerung: $\frac{N-1}{2}$ Samples

Low-Pass / High-Pass (1. Ordnung):

$$y[n] = \alpha \cdot x[n] + (1-\alpha) \cdot y[n-1]$$

- $\alpha \in (0, 1]$: Glättungsfaktor
- α klein $\Rightarrow$ starke Glättung
- High-Pass: $y_{\mathrm{HP}} = x - y_{\mathrm{LP}}$

Implementierung (C)

```c
float lp_filter(float x, float y_prev, float alpha) {
    return alpha * x + (1.0f - alpha) * y_prev;
}
```

Ziel

Optimale Zustandsschätzung unter Berücksichtigung von Prozessrauschen und Messrauschen.

Zwei Schritte (rekursiv):

1. Prädiktion: Vorhersage des nächsten Zustands anhand des Systemmodells

$$\hat{x}_{k|k-1} = A\,\hat{x}_{k-1} + B\,u_k, \quad P_{k|k-1} = A\,P_{k-1}\,A^T + Q$$

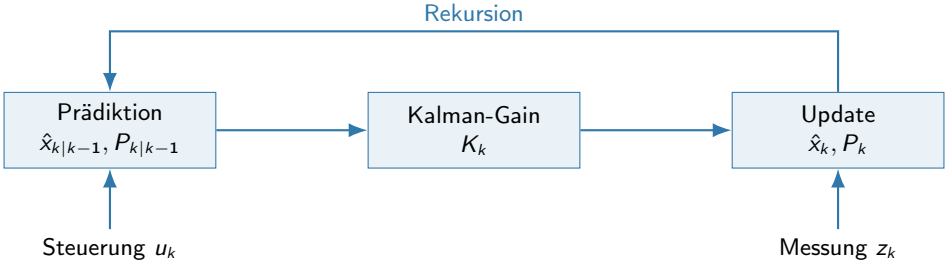
2. Update: Korrektur mit der aktuellen Messung

$$K_k = P_{k|k-1}\,H^T\,(H\,P_{k|k-1}\,H^T + R)^{-1}$$

$$\hat{x}_k = \hat{x}_{k|k-1} + K_k\,(z_k - H\,\hat{x}_{k|k-1}), \quad P_k = (I - K_k H)\,P_{k|k-1}$$

Q: Prozessrausch-Kovarianz, R: Messrausch-Kovarianz, K_k: Kalman-Gain

Kalman-Filter – Ablauf



Intuition: Der Kalman-Gain K_k gewichtet, ob man mehr dem *Modell* oder der *Messung* vertraut – je nach Unsicherheit.

Complementary Filter (IMU)

Problem: Gyroskop liefert gute *kurzfristige* Orientierung (aber driftet), Beschleunigungssensor liefert stabile *langfristige* Neigung (aber verrauscht).

Complementary Filter

$$\theta_k = \alpha \cdot (\theta_{k-1} + \omega_k \cdot \Delta t) + (1 - \alpha) \cdot \theta_{\text{acc,k}}$$

mit $\alpha \approx 0.98$ (High-Pass auf Gyro, Low-Pass auf Beschleunigung).

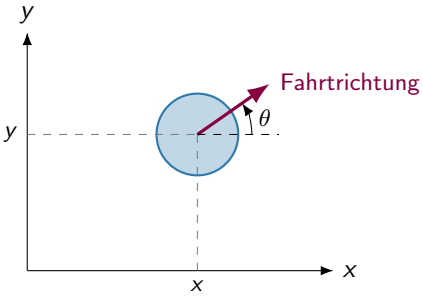
- Vorteil: Extrem einfach, kaum Rechenaufwand
- Nachteil: Nicht optimal, kein Modell der Unsicherheit
- Einsatz: Drohnen, Turtlebot, schnelle Prototypen

Zustandsschätzung

Pose eines Roboters

Die Pose beschreibt Position und Orientierung in der Ebene:

$$\mathsf{q} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \in \mathbb{R}^2 \times (-\pi, \pi]$$



Warum reicht (x, y) nicht?

- Roboter muss wissen, *wohin* er schaut
- Steuerung hängt von θ ab
- Sensoren sind richtungsabhängig (Kamera, Lidar)

Odometrie – Wheel Encoder

Idee: Aus der Raddrehung auf die zurückgelegte Strecke schließen.

Encoder-Modell:

- Ticks pro Umdrehung: N_{ticks}
- Radradius: r
- Strecke pro Tick: $\Delta s = \frac{2\pi r}{N_{\text{ticks}}}$

Differentielle Odometrie:

$$\Delta s = \frac{\Delta s_L + \Delta s_R}{2}$$

$$\Delta\theta = \frac{\Delta s_R - \Delta s_L}{L}$$

(L = Achsabstand)

Problem: Drift!

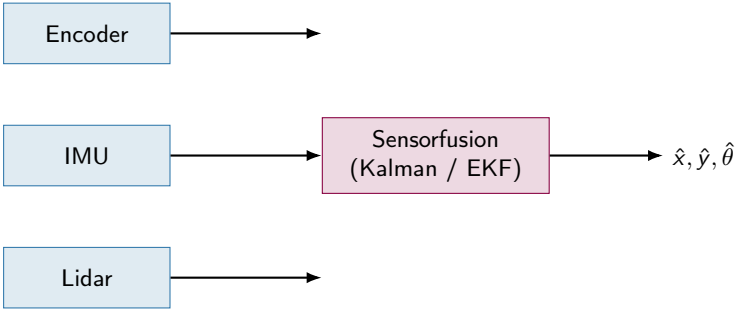
Odometrie akkumuliert Fehler über die Zeit:

- Radschlupf
- Ungleiche Raddurchmesser
- Bodenunebenheiten

$\Rightarrow$ Sensorfusion nötig!

Sensorfusion

Ziel: Mehrere Sensoren kombinieren, um die Schwächen einzelner Sensoren auszugleichen.



- Encoder: hochfrequent, aber driftet
- IMU: schnelle Dynamik, aber Gyro-Drift
- Lidar: absolut genau, aber niedrige Rate

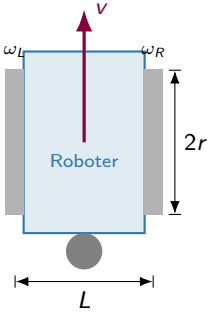
$\Rightarrow$ Kalman-Filter gewichtet je nach Unsicherheit.

Kinematik & Bewegungsmodelle

Differential Drive (Turtlebot)

Aufbau:

- Zwei unabhängig angetriebene Räder
- Stützrad (passiv)
- Parameter: Radradius r, Achsabstand L



Geschwindigkeiten:

$$v = \frac{r}{2}(\omega_R + \omega_L)$$
$$\omega = \frac{r}{L}(\omega_R - \omega_L)$$

ω_R, ω_L: Winkelgeschwindigkeiten der Räder

Forward & Inverse Kinematics

Forward Kinematics:
$v, \omega \;\rightarrow\; x, y, \theta$

Diskretisiert (Δt klein):

$$
\begin{aligned}
x_{k+1} &= x_k + v_k \cos(\theta_k)\,\Delta t \\
y_{k+1} &= y_k + v_k \sin(\theta_k)\,\Delta t \\
\theta_{k+1} &= \theta_k + \omega_k\,\Delta t
\end{aligned}
$$

Gegeben: Rad-Geschwindigkeiten
Gesucht: resultierende Pose

Inverse Kinematics:
Gewünschte Bewegung $\rightarrow$
Radgeschwindigkeiten

$$
\omega_R = \frac{v + \frac{L}{2}\,\omega}{r}
$$

$$
\omega_L = \frac{v - \frac{L}{2}\,\omega}{r}
$$

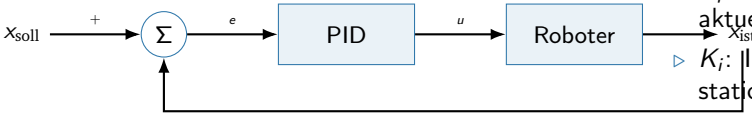
Gegeben: gewünschtes (v, ω)
Gesucht: Motoransteuerung (ω_L, ω_R)

Regelung (Control)

PID-Regelgesetz

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}$$

mit Regelabweichung $e(t) = x_{\text{soll}}(t) - x_{\text{ist}}(t)$



- K_p: Proportional – reagiert auf aktuellen Fehler
- K_i: Integral – beseitigt stationären Fehler
- K_d: Differential – dämpft Überschwinger

Trajektorienverfolgung

Aufgabe: Roboter soll einem Pfad folgen oder einen Zielpunkt (x_g, y_g) erreichen.

Einfache Strategie ("go-to-goal"):

1. Berechne Richtung zum Ziel: $\alpha = \text{atan2}(y_g - y,\ x_g - x)$
2. Regelfehler Orientierung: $e_\theta = \alpha - \theta$
3. PID auf e_θ liefert ω
4. v konstant oder abhängig von Distanz

Stabilität

- Overshoot: Roboter schießt über Ziel hinaus (K_p zu groß)
- Oszillation: Pendeln um Zielpunkt (K_d zu klein)
- Stationärer Fehler: Ziel wird nie ganz erreicht ($K_i = 0$)

Lokalisierung & Mapping (SLAM)

Das Lokalisierungsproblem

Zwei fundamentale Fragen:

1. Wo bin ich? (Lokalisierung)
2. Wie sieht die Welt aus? (Mapping)

Henne-Ei-Problem:

- Lokalisierung braucht Karte
- Kartenbau braucht Position
- $\Rightarrow$ beides gleichzeitig: SLAM

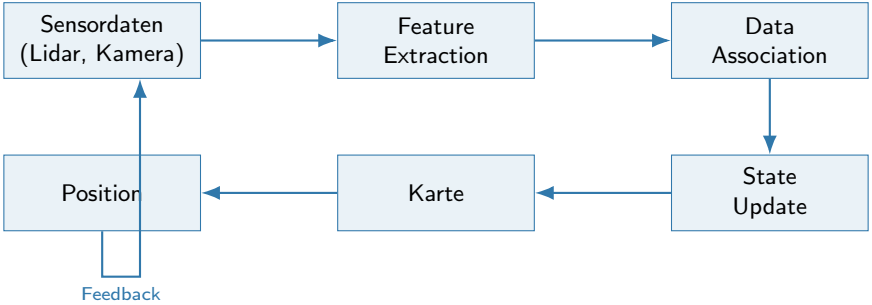
Landmarken:

- Markante Punkte in der Umgebung
- Wiedererkennbar (Ecken, Kanten, Objekte)
- Distanz + Winkel $\Rightarrow$ Triangulation

Occupancy Grid Map:

- Raster: jede Zelle frei / belegt / unbekannt
- Aufgebaut aus Lidar-Scans
- Grundlage für Pfadplanung

SLAM – Simultaneous Localization and Mapping



- Feature Extraction: Landmarken aus Sensordaten extrahieren
- Data Association: Welche beobachtete Landmarke entspricht welcher bekannten?
- State Update: Karte und Position gemeinsam korrigieren (EKF-SLAM, Partikelfilter, Graph-SLAM)

Pfad- und Trajektorienplanung

Pfadplanung – A* und Dijkstra

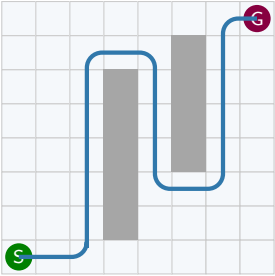
Gegeben: Occupancy Grid Map, Start s, Ziel g.
Gesucht: Kürzester hindernisfreier Pfad.

Dijkstra:

- Expandiert alle Knoten nach Kosten
- Garantiert optimalen Pfad
- Keine Richtungsinformation
  $\Rightarrow$ langsam bei großen Karten

A*:

- Erweitert Dijkstra um Heuristik $h(n)$
- $f(n) = g(n) + h(n)$
- Typisch: $h =$ euklidische Distanz zum Ziel
- Optimal, wenn h zulässig (nicht überschätzt)



Trajektorienplanung & Hindernisvermeidung

Pfad $\neq$ Trajektorie:

- Pfad: geometrische Kurve $(x(s), y(s))$
- Trajektorie: Pfad + Zeitparametrisierung $(x(t), y(t), \theta(t))$

Anforderungen an die Trajektorie:

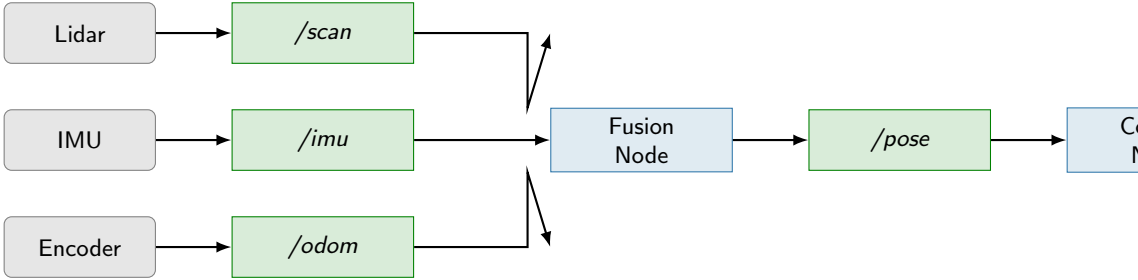
1. Glatt: stetig differenzierbar (keine Sprünge in v oder ω)
2. Physikalisch realisierbar: $v_{\max}$, $\omega_{\max}$ einhalten
3. Hindernisfrei: Sicherheitsabstand zu Objekten

Hindernisvermeidung (reaktiv):

- Bug-Algorithmen: Hindernis umfahren, dann Zielrichtung wieder aufnehmen
- Potentialfeldmethode: Ziel = anziehend, Hindernisse = abstoßend
- Dynamic Window Approach (DWA): Suche im (v, ω)-Raum

Softwarearchitektur (ROS 2)

Sensor → Topic → Node → Control Loop



Jede Kante ist ein ROS 2 Topic – lose Kopplung, austauschbare Komponenten.

Publisher / Subscriber:

- Asynchrone Kommunikation
- Publisher sendet auf Topic
- Subscriber empfängt Callback
- Kein direkter Aufruf ⇒ Entkopplung

Quality of Service (QoS):

- `RELIABLE`: jede Nachricht kommt an (TCP-ähnlich)
- `BEST_EFFORT`: keine Garantie (UDP-ähnlich, weniger Latenz)
- `KEEP_LAST(n)`: nur die *n* neuesten Nachrichten puffern

Wichtig für Echtzeit!

Falsche QoS-Einstellungen können zu:

- verlorenen Sensordaten
- erhöhter Latenz
- nicht-deterministischem Verhalten

führen. ⇒ QoS bewusst wählen!

Typische Kombination:

- Sensordaten: `BEST_EFFORT`
- Steuerbefehle: `RELIABLE`

Timing: Frequenzen und Latenz

Verschiedene Raten im System:

Komponente	Rate	Typ
IMU	200 Hz	Sensor
Lidar	10 Hz	Sensor
Kamera	30 Hz	Sensor
Odometrie	50 Hz	Verarbeitung
Regelung	20 Hz	Control
SLAM	1 Hz	Planung

Latenz-Budget:

- Gesamtlatenz: Sensor $\rightarrow$ Aktor
- Muss kleiner als Regelungsperiode sein
- Bei 20 Hz Control: Budget < 50 ms

ROS 2 Timer

```
timer = create_wall_timer(
  50ms, callback);
```

Echtzeit und Embedded-Aspekte

Sampling Rates vs. Control Rates

Kernunterscheidung:

- Sampling Rate: Wie oft wird gemessen? (Sensorfrequenz)
- Control Rate: Wie oft wird geregelt? (Aktorfrequenz)

In der Regel gilt: $f_{\text{sensor}} \geq f_{\text{control}} \geq f_{\text{planning}}$

Beispiel Turtlebot

IMU Sampling:	100 Hz
Encoder Sampling:	50 Hz
PID Control Loop:	20 Hz
Pfadplanung:	1 Hz

Jede Schicht konsumiert die Daten der darüberliegenden.

Determinismus, Interrupts, Polling

Determinismus:

- Regelschleife muss garantiert innerhalb der Deadline fertig sein
- Linux ist *nicht* echtzeitfähig (aber: PREEMPT_RT Patch)
- Microcontroller: oft bare-metal oder RTOS

Interrupts vs. Polling:

- Interrupt: Hardware signalisiert Ereignis ⇒ sofortige Reaktion
- Polling: Software fragt zyklisch ab ⇒ einfacher, aber Latenz

Hardware-nahe Verarbeitung (C):

- Direkte Registerzugriffe
- Kein dynamisches Speichermanagement
- Vorhersagbare Ausführungszeit

```c
// Interrupt Service Routine
void IMU_IRQHandler(void) {
    int16_t raw = IMU->DATA_REG;
    ring_buffer_push(&imu_buf, raw);
    // Keine malloc, printf etc.!
}

// Polling im Main-Loop
while (1) {
    if (timer_elapsed(20ms)) {
        run_pid_controller();
    }
}

```

Unsicherheit & Verifikation

Umgang mit Unsicherheit

In der Robotik ist nichts exakt:

- Sensoren rauschen (Gaussian Noise, Bias)
- Modelle sind Approximationen (Radschlupf, Linearisierung)
- Die Umgebung ist dynamisch und teilweise unbekannt

Konsequenz für den Entwurf:

1. Probabilistische Modelle: Zustand als Wahrscheinlichkeitsverteilung, nicht als Punkt
2. Robuste Regelung: Sicherheitsmargen in Geschwindigkeit und Abstand
3. Defensive Planung: Worst-Case-Szenarien einplanen

Safety-Anforderung (Beispiel)

"Der Roboter hält *immer* mindestens 30 cm Abstand zu jedem Hindernis" – unabhängig von Sensorrauschen.

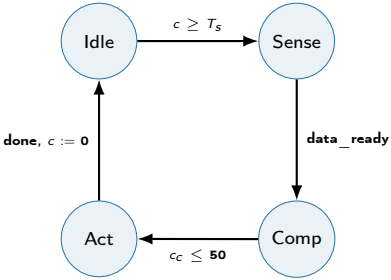
Rückgriff auf die Modellierungstechniken der Vorlesung!

Timed Automata für Control Loops:

- Zustände: `Idle`, `Sensing`, `Computing`, `Actuating`
- Uhren: c_{sense}, c_{ctrl}
- Guards: $c_{\text{ctrl}} \leq 50\,\text{ms}$
- Invarianten erzwingen Deadlines

Safety-Eigenschaften:

- "Control-Schleife terminiert immer innerhalb von 50 ms"
- "Sensordaten sind nie älter als 100 ms"



$c_C \leq 50$: Invariante

Verifikation mit UPPAAL

UPPAAL (bekannt aus der Vorlesung) eignet sich hervorragend für die Verifikation von Robotik-Scheduling:

Modellierung:

- Jede Task (Sensor, Filter, Control) als Timed Automaton
- Kommunikation über Channels
- Shared Variables für Sensordaten

Verifikation (CTL):

- `A[] not deadlock`
  (kein Deadlock möglich)
- `A[] ctrl.active imply c_ctrl <= 50`
  (Deadline wird immer eingehalten)
- `A[] distance >= 30`
  (Sicherheitsabstand garantiert)

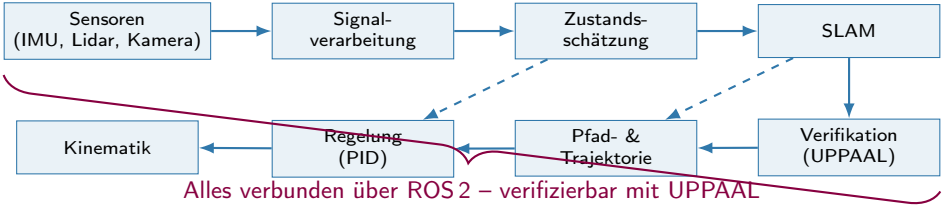
Bezug zur Vorlesung

- FSMs → Verhalten
- Timed Automata → Timing
- UPPAAL → Verifikation
- C / Rust → Implementierung
- ROS 2 → Integration

Alle Werkzeuge kommen hier zusammen!

Zusammenfassung

Zusammenfassung: Von Sensoren zur sicheren Robotik



Kernbotschaften

1. Sensoren sind ungenau – Rauschen, Bias und Drift müssen modelliert und gefiltert werden.
2. Kalman-Filter ist das zentrale Werkzeug für Zustandsschätzung unter Unsicherheit.
3. Kinematik verbindet Steuergrößen mit Bewegung – Forward und Inverse Kinematics.
4. PID-Regelung ist der Arbeitspferd-Regler; Stabilität muss beachtet werden.
5. SLAM löst das Henne-Ei-Problem der gleichzeitigen Lokalisierung und Kartierung.
6. Echtzeit ist kein Nice-to-have: Deadlines müssen eingehalten werden.
7. Formale Verifikation (Timed Automata, UPPAAL) schließt den Kreis zur sicheren Implementierung.